

The Network Layer: The Control Plane

Lecture 20



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



Network Layer

Routing: Introduction

- We recall that, on packet receipt, a **router can perform 3 actions**:
 - **Forwarding the packet** as is to a suitable neighbor.
 - **Forwarding a modified/replaced packet** (e.g., fragmentation, masquerading, etc.).
 - **Dropping the packet** (e.g., if expired).
- There are **different types of routers**, some are very simple and are used by local administrators (for instance to connect LANs and WANs), others are used by ISPs to allow packets to be forwarded to the right device in the entire Internet.
- As we saw, **routers are equipped with forwarding tables** that specify the local forward strategy (in the neighborhood of the router). The way these tables are created can be centralized or decentralized.
 - Routers have no knowledge about the rest of the network (at least on startup).

Network Layer

Routing: Introduction

- To decide where packets should be forwarded routers make use of **routing algorithms**, which determine good paths (i.e., sequence of routes) from senders to receivers.
- Typically, a **“good” path is one that has the minimal cost** (shortest or fastest path, etc.), in real-world **there are also policy issues** to be considered (e.g., rules specifying if an organization can talk with another etc.).
- The **routing problem is crucial for computer networks**, but the general problem of finding “good paths” on a network (or a graph) is particularly relevant not only for computer networks (e.g., videogames, autonomous driving, robotics, etc.).

Network Layer

Routing: Flooding

- Most simple approach is **flooding**: each router forwards all incoming packets to all links except for the one by which the packet is received.
 - All paths are explored (optimal one included), so the **packet will surely be delivered** if a path exists.
 - Packets going nowhere will eventually reach their **hop limit** and be discarded.
 - It is **tremendously robust** due to the redundancy of generated packets (e.g., if some routers are offline alternative paths are always considered).
- Flooding can be used in specific situations where redundancy is needed (e.g., broadcast of messages, military scenarios, etc.) but it is **impractical in high-performance and large networks** (such as Internet).
- For typical computer networks a more sophisticated routing algorithm is needed.

Network Layer

Routing: Problem Formulation

- We can **formalize** our network as a graph $G=(N,E)$, where:
 - N is a set of **nodes** (the routers of our network).
 - E is a set of **edges** (physical links) represented as a pair of nodes.
- An **edge** also has a **value** representing its **cost**, which may reflect the physical length of the link, the link speed, or the monetary cost associated with a link.
- We can formalize a generic **cost** as a function c such that, given a generic couple of nodes (x,y) :
 - If $(x,y) \in E$, then $c(x,y)$ is the cost of the edge (x,y)
 - If $(x,y) \notin E$, then $c(x,y) = \infty$
 - If $(x,y) \in E$, then also $(y,x) \in E$ (undirected graph) and $c(x,y) = c(y,x)$

Network Layer

Routing: Problem Formulation

- Following this formulation, a **path** is a **sequence of nodes** such that **all adjacent nodes are linked**, formally:

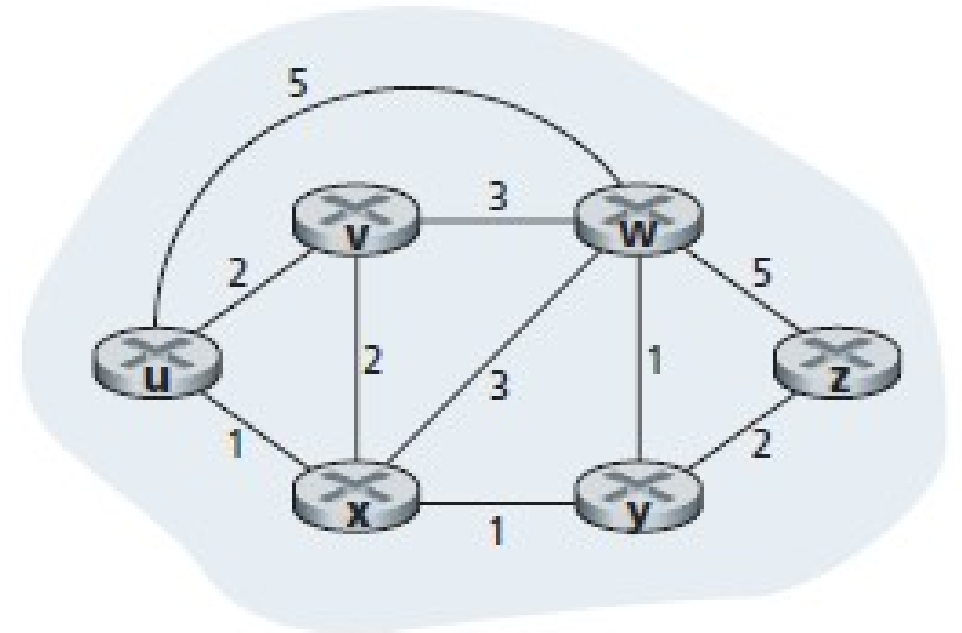
$$(x_1, x_2, \dots, x_p) \text{ with } (x_1, x_2), \dots, (x_{p-1}, x_p) \in E$$

- The **overall cost of the path** is the sum of the links' costs
- Let's call the **set of all possible paths** connecting the nodes u and z , we can define the **best path** as the path having **minimum cost**.

Network Layer

Routing: Problem Formulation

- In this example we have a graph composed of 6 nodes
- For instance, the best path between nodes u and w is: [exercise]



Network Layer

Routing: Problem Formulation

- Algorithms able to find an optimal path between two nodes are called **least-cost path algorithms** (**shortest-path** in case of the same cost between all pairs of nodes).
- In the case of computer networks, we are interested in **finding such optimal path** not for just two nodes but **for all possible couples of nodes**.
- A routing algorithm **converges** when the shortest path for all couples of nodes in the network is found.
- On networks **there are 2 families of algorithms** that are typically used:
 - **Distance Vector.**
 - **Link State.**

Network Layer

Routing: Distance-Vector Algorithm

- The **Distance-Vector** (DV) algorithm is a **decentralized** approach that exploit **local knowledge** about the network combined with nodes communication to find the shortest paths.
 - It is based on the **Bellman-Ford algorithm** that is used to compute paths.
- Here **each node receives some information** from one or more of its directly attached neighbors, performs a calculation, and then **distributes the results** of its calculation back to its neighbors.
- This algorithm is **asynchronous** in that it does not require all of the nodes to be coordinated with each other.

Network Layer

Routing: Distance-Vector Algorithm

Let's denote with $d_x(y)$ the cost of the best path from node x to node y . From the **Bellman equation** we can derive the following one:

$$d_x(y) = \min_v \{c(x,v) + d_v(y)\}$$

This is quite intuitive: **there must be an adjacent node v to x that lies on the best path from x to y** , and the cost of the said best path is the cost of reaching v plus the best path between v and y .

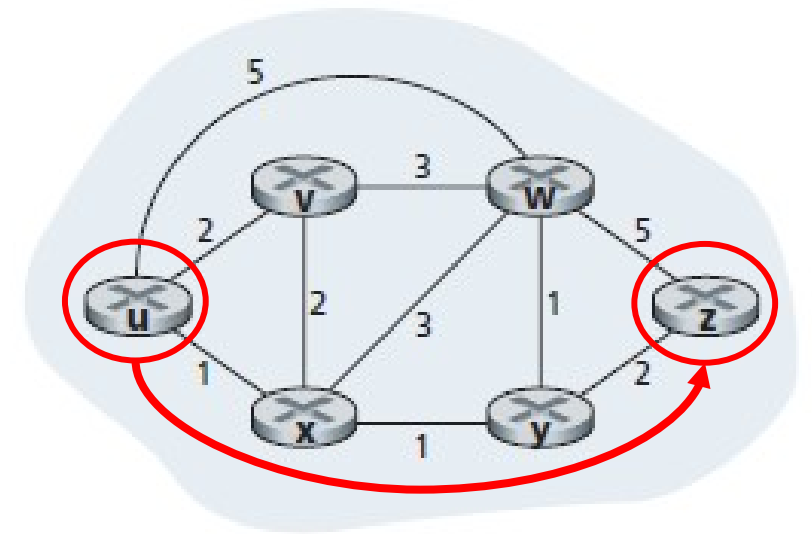
- If we are able to find such v **we can just add it into the forwarding table** and forward to it all packets directed to .
- To do so, **we should have an estimation of** for all neighbors (the distance vector).

Network Layer

Routing: Distance-Vector Algorithm

- Let's try with a real example. Considering u and z, the best path is

$u \rightarrow x \rightarrow y \rightarrow z$

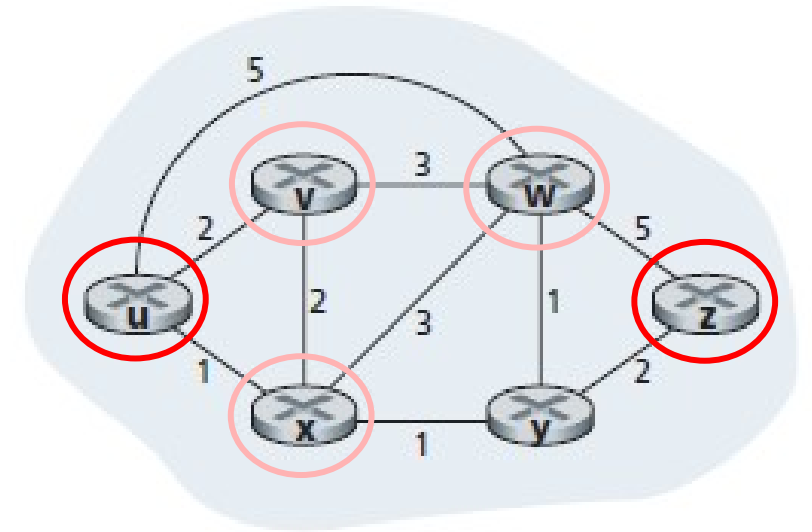


Network Layer

Routing: Distance-Vector Algorithm

- Now, for all the nodes along this optimal path the **Bellman equation must hold**.
 - Let's consider **node u**, which has 3 neighbours w, v, and x
 - For x we have $c(u,x)=1$ and $d_x(z)=3$.
 - For v we have $c(u,v)=2$ and $d_v(z)=5$.
 - For w we have $c(u,w)=5$ and $d_w(z)=3$

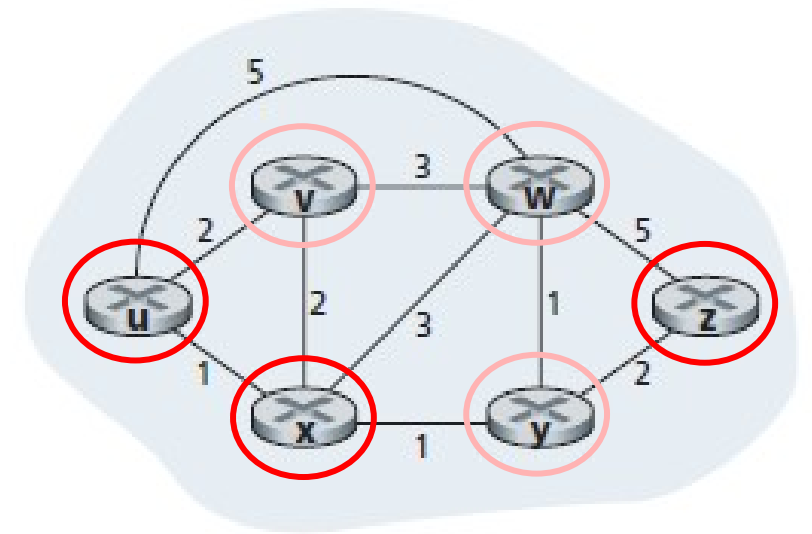
Node x minimises the expression $c(u,x)+d_x(z)$, therefore it is the next one on the optimal path.



Network Layer

Routing: Distance-Vector Algorithm

- We can **iterate this process on the next node** of the path (which is x) and we will see that **the Bellman equation stands** (i.e., that node y, which is next in the path, will also minimize the expression, and so on).
- If we have the distance vector for all nodes, **we can easily use Bellman equation to compute the next hop** in the path (forwarding table).



Network Layer

Routing: Distance-Vector Algorithm

- The pseudocode:

```
DistanceVector(x)
  for all nodes v
    if v is a neighbor of x then
       $D_x(v) = c(x,v)$ 
    else
       $D_x(v) = \infty$ 
  for all neighbors w and destinations y
     $D_w(y) = ?$ 
  send initial  $D_x(\cdot)$  to all neighbors

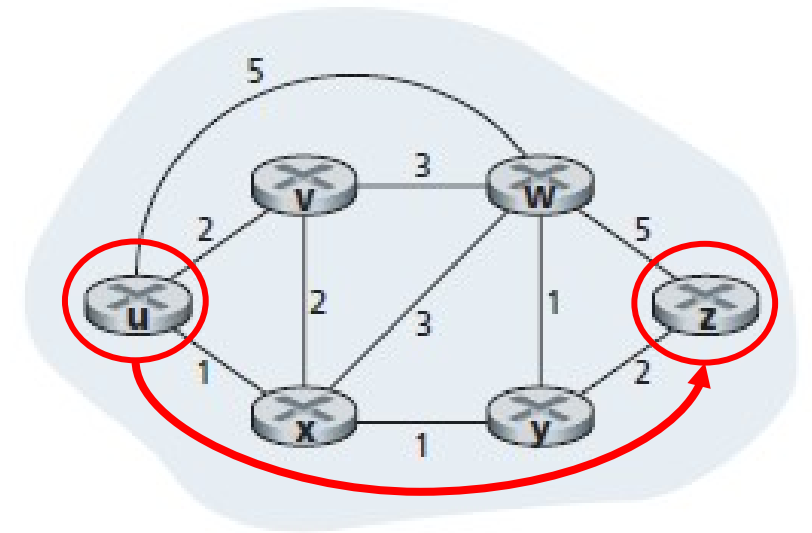
  repeat
    if cost of a neighbor updated then
      for all nodes y
         $D_x(y) = \min_v \{c(x,v) + D_v(y)\}$ 
        if  $D_x(y)$  changed for any destination y then
          send updated  $D_x(\cdot)$  to all neighbors
  until false //forever
```

- We use the **data-structure** $D_w(v)$ to store the distance vectors from all nodes w to target nodes v.
- During the **initialization** phase:
 - Only **distances to neighbors** are set.
- During the **online** phase:
 - The **news** from the neighborhood are received.
 - Distance vectors are **updated** (Bellman equation).
 - Local changes are **communicated** to adjacent nodes.

Network Layer

Routing: Distance-Vector Algorithm

- To clarify this process **let's focus again on the path between u and z:**
 1. On start, **u only knows its neighbors**. The cost to z is infinity ($D_u(z) = \infty$).
 2. Node **y wakes up** and discovers a better path to z, which is the direct link y-z with cost 2 ($D_y(z) = c(y,z) = 2$). It communicates the good news to x.
 3. Node **x discovers that the best path to z now passes through y** with cost 3 ($D_x(z) = c(x,y) + D_y(z) = 1 + 2$), and forwards the good news to u.
 4. Node **u receives the news**. Now there is a path to z that costs less than infinity passing through x ($D_u(z) = c(u,x) + D_x(z) = 1 + 3$).
- The **same process** takes place for all nodes/paths.



Network Layer

Routing: Distance-Vector Algorithm

- As for the computational complexity, every time the cost of a node is updated, all nodes have to re-evaluate their edges. This process leads to a **quadratic worst-case complexity**.
- Pros:
 - The **algorithm** is **asynchronous**, this facilitates the computation as routers can on-line and in any stage adapt to updated costs.
- Cons:
 - **Slow convergence**: updates spread slowly as all nodes have to detect the change and send an updated to the adjacent ones.
 - **Count-to-infinity**: while improved paths are immediately recognized and adopted by the network, failures are detected slowly and may produce loops.

Network Layer

Routing: Count-to-infinity

```
DistanceVector(x)
  for all nodes v
    if v is a neighbor of x then
       $D_x(v) = c(x,v)$ 
    else
       $D_x(v) = \infty$ 
  for all neighbors w and destinations y
     $D_w(y) = ?$ 
  send initial  $D_x(\cdot)$  to all neighbors

  repeat
    if cost of a neighbor updated then
      for all nodes y
         $D_x(y) = \min_v \{c(x,v) + D_v(y)\}$ 
      if  $D_x(y)$  changed for any destination y then
        send updated  $D_x(\cdot)$  to all neighbors
  until false //forever
```

- The **count-to-infinity** problem is due to the “local” nature of the DV algorithm (which is also its power).
- The local vector ($D_x(y)$) is directly computed **starting from the vectors of other nodes** ($D_v(y)$).
- There is **no indication about which nodes are in the best paths** chosen by adjacent nodes.
 - Can I be sure that I’m not in the “best path” of my neighbors?

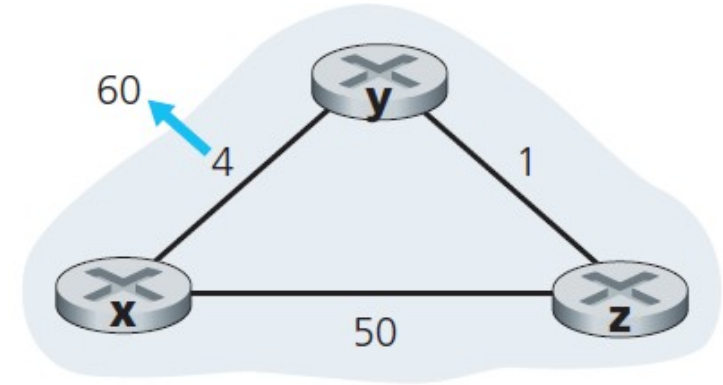
Network Layer

Routing: Count-to-infinity

- Let's consider this simplified example in which **the link x-y gets a sudden increment of cost and only y notice it!**

- $D_y(x) = 4$,
- $D_y(z) = 1$,
- $D_x(y) = 4$,
- $D_x(z) = 4 + D_y(z) = 5$,
- $D_z(x) = 1 + D_y(x) = 5$,
- $D_z(y) = 1$.

This is the **initial situation** before the increment ($c(x,y)$ is still 4).



Network Layer

Routing: Count-to-infinity

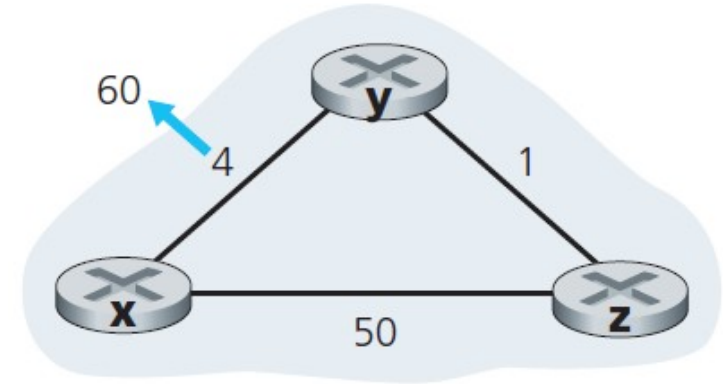
- Let's consider this simplified example in which **the link x-y gets a sudden increment of cost and only y notice it!**

- $D_y(x) = 4,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 5,$
- $D_z(y) = 1.$

Node y detects the increment and updates distance-vector to x:

$$D_y(x) = \min\{ 60, 1 + D_z(x) \} = 1 + D_z(x) = 6,$$

- $D_y(x) = 1 + D_z(x) = 6,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 5,$
- $D_z(y) = 1.$



Network Layer

Routing: Count-to-infinity

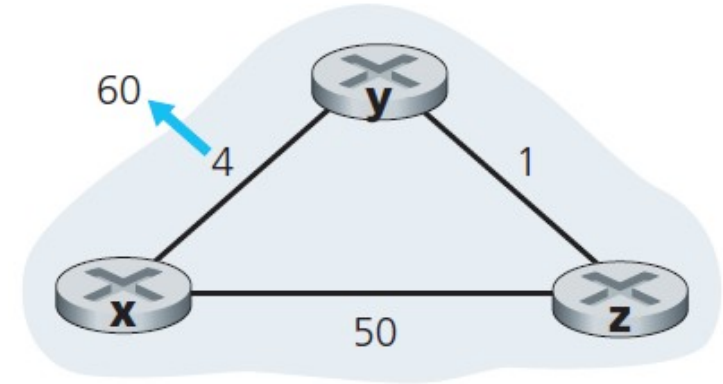
- Let's consider this simplified example in which **the link x-y gets a sudden increment of cost and only y notice it!**

- $D_y(x) = 4,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 5,$
- $D_z(y) = 1.$

But vector $D_y(x)$ is used by z to compute the path to x, so also $D_z(x)$ must be updated!

- $D_y(x) = 1 + D_z(x) = 6,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 5,$
- $D_z(y) = 1.$

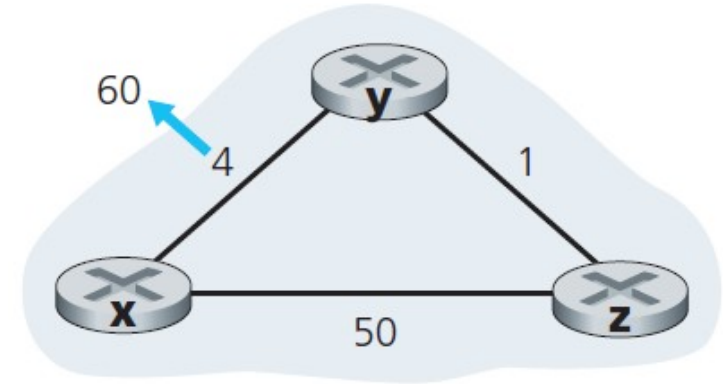
- $D_y(x) = 1 + D_z(x) = 6,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 7,$**
- $D_z(y) = 1.$



Network Layer

Routing: Count-to-infinity

- Let's consider this simplified example in which **the link x-y gets a sudden increment of cost and only y notice it!**



- $D_y(x) = 4,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 5,$
- $D_z(y) = 1.$

But vector $D_z(x)$ is itself used by y to compute the path to x , so $D_y(x)$ must be updated again!

- $D_y(x) = 1 + D_z(x) = 6,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 5,$
- $D_z(y) = 1.$

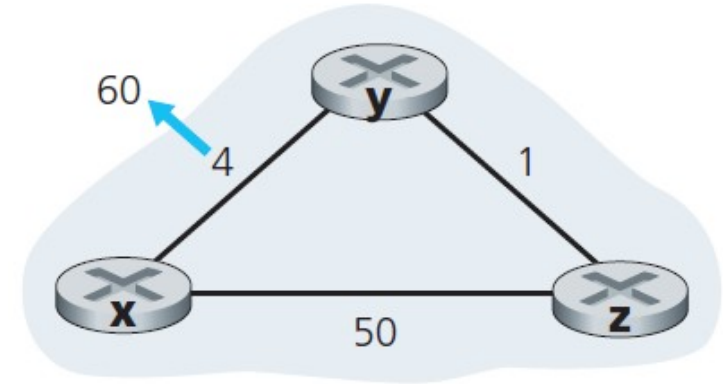
- $D_y(x) = 1 + D_z(x) = 6,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 7,$
- $D_z(y) = 1.$

- $D_y(x) = 1 + D_z(x) = 8,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 7,$
- $D_z(y) = 1.$

Network Layer

Routing: Count-to-infinity

- Let's consider this simplified example in which **the link x-y gets a sudden increment of cost and only y notice it!**



- $D_y(x) = 4,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 5,$
- $D_z(y) = 1.$

This is a loop: the 2 vectors will be continuously updated until a stable configuration is reached.

- $D_y(x) = 1 + D_z(x) = 6,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 5,$
- $D_z(y) = 1.$

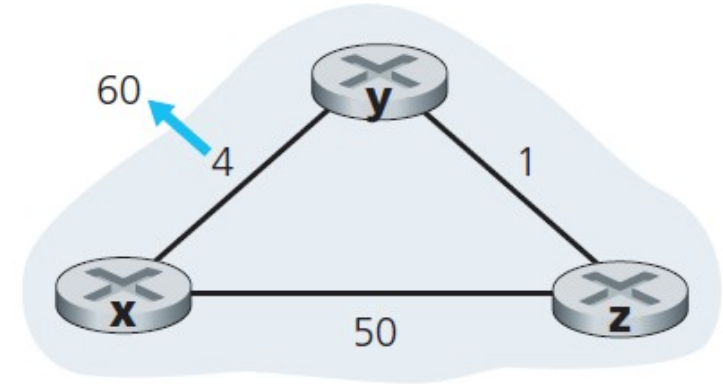
- $D_y(x) = 1 + D_z(x) = 6,$
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 9,$**
- $D_z(y) = 1.$

- $D_y(x) = 1 + D_z(x) = 8,$**
- $D_y(z) = 1,$
- $D_x(y) = 4,$
- $D_x(z) = 4 + D_y(z) = 5,$
- $D_z(x) = 1 + D_y(x) = 7,$
- $D_z(y) = 1.$

Network Layer

Routing: Count-to-infinity

- This problem could have been avoided if x had also noticed/communicated the change of the x-y cost.
 - If x is malfunctioning or delayed for some reason (e.g., the high costs of x's links may indicate that x is malfunctioning), it will take time to converge.



- $D_y(x) = \text{MIN}\{ c(y,x), c(y,z) + D_z(x) \} = c(y,z) + D_z(x) = 1 + 50 = 51,$
- $D_y(z) = \text{MIN}\{ c(y,z), c(y,x) + D_x(z) \} = c(y,z) = 1,$
- $D_x(y) = \text{MIN}\{ c(x,y), c(x,z) + D_z(y) \} = c(x,z) + D_z(y) = 50 + 1 = 51,$
- $D_x(z) = \text{MIN}\{ c(x,z), c(x,y) + D_y(z) \} = c(x,z) = 50,$
- $D_z(x) = \text{MIN}\{ c(z,x), c(z,y) + D_y(x) \} = c(z,x) = 50,$
- $D_z(y) = \text{MIN}\{ c(z,y), c(z,x) + D_x(y) \} = c(z,y) = 1.$

Costs convergence in case node x detects the change.